**Universidad Autónoma de Baja California**

*Facultad de Ciencias Quimicas e Ingenieria*

Ingeniería en Computación

**Navarrete Pérez Andrés - 1284707**

Proyecto de carrera

Taller 7



## 1. Propósito y alcance de los modelos

ChiTito Smart Dispenser prepara una porción de alimento para Chiquin o Tito a partir de una solicitud realizada en la aplicación. El ESP32 mide el alimento servido con una celda de carga y HX711, controla una compuerta de dosificación y permite acceder al plato únicamente al tag RFID de la mascota seleccionada. Además, supervisa el alimento restante, el nivel de agua y su condición, y controla una bomba de recirculación.

Este documento describe los actores, casos de uso, actividades, mensajes y estados necesarios para implementar ese comportamiento. La arquitectura conserva la alternativa 1 seleccionada en el Taller 4: ESP32, Wi-Fi, MQTT con Mosquitto, Laravel, MySQL y Vue 3 como PWA [1]. El control principal se solicita desde la aplicación; las decisiones inmediatas de seguridad se ejecutan en el ESP32.

Se adopta el criterio de seguridad acordado para la desconexión: bloquear nuevas acciones remotas, cerrar el dosificador, apagar la bomba y pausar el acceso al plato. Continúan las lecturas necesarias y los reintentos de conexión. Si el acceso está abierto, se ordena un cierre gradual acorde con el diseño mecánico. La reconexión recupera la información y requiere una solicitud nueva o una revalidación explícita para volver a actuar.

Los modelos son una especificación de diseño. Los tiempos, límites y tolerancias de control deberán validarse con el prototipo; este reporte no presenta mediciones experimentales ni resultados de pruebas ya realizadas.

### 1.1 Componentes y responsabilidades

| Componente | Responsabilidad dentro del sistema |
|---|---|
| ESP32 DevKit V1 | Ejecutar control local, validar comandos, leer sensores y manejar estados seguros. |
| HX711 y celda de carga de 5 kg | Medir el peso del plato de alimento y calcular la masa agregada. |
| VL53L1X y A02YYUW | Estimar el alimento restante y el agua disponible con calibración geométrica. |
| RC522 y dos tags | Identificar el UID del collar y compararlo con la mascota autorizada. |

| Componente | Responsabilidad dentro del sistema |
|---|---|
| Dos servos MG90S | Servo 1 para dosificar; servo 2 para abrir o cerrar la protección del plato. |
| SEN0244 y SEN0189 | Observar cambios de TDS y turbidez para avisos de mantenimiento. |
| Bomba de 5 V y AO3400A | Ordenar recirculación o parada con protección por nivel de agua. |
| Mosquitto y Laravel | Transportar mensajes y aplicar validación, permisos, persistencia y API. |
| MySQL y Vue 3 PWA | Conservar historial y mostrar controles, resultados, lecturas y alertas. |

### 1.2 Límites funcionales

La masa servida es la diferencia entre el peso estable final y el peso inicial del plato. El registro no representa necesariamente lo que la mascota consumió. El sistema de agua recircula y monitorea disponibilidad; no mide una ración exacta de agua ni el consumo por mascota. Los sensores de TDS y turbidez permiten observar cambios, pero no certifican potabilidad [5], [6].

La cámara, la inteligencia artificial, el análisis microbiológico, el cambio automático del agua y una aplicación nativa independiente permanecen fuera del alcance inicial [1]. El HX711 se utiliza en el plato de alimento. No se agregan caudalímetros, finales de carrera ni sensores de presencia a los componentes seleccionados.

## 2. Requisitos funcionales utilizados en la trazabilidad

Los siguientes identificadores se asignan en este reporte para relacionar requisitos, casos de uso y modelos. No se presentan como una numeración previa del proyecto.

| ID | Requisito funcional |
|---|---|
| RF01 | Autenticar al usuario y comprobar permisos sobre el dispositivo. |
| RF02 | Registrar Chiquin y Tito y asociar un UID RFID distinto a cada mascota. |
| RF03 | Seleccionar mascota y cantidad de alimento desde la aplicación. |
| RF04 | Enviar órdenes de control desde frontend hasta el ESP32 por el backend. |
| RF05 | Dosificar por peso y cerrar la salida al alcanzar el umbral de cierre. |
| RF06 | Registrar gramos solicitados, gramos realmente servidos y resultado. |
| RF07 | Autorizar el acceso al plato únicamente al UID de la mascota seleccionada. |
| RF08 | Estimar y mostrar el alimento disponible en el depósito. |
| RF09 | Estimar el agua disponible y bloquear la bomba por nivel bajo o inválido. |
| RF10 | Activar y detener la recirculación del agua desde la aplicación. |
| RF11 | Monitorear TDS y turbidez y avisar cambios en la condición del agua. |
| RF12 | Publicar telemetría y consultar lecturas mediante la API del backend. |
| RF13 | Consultar historial de porciones, comandos y eventos por mascota. |
| RF14 | Generar alertas de recursos bajos, fallas y pérdida de conexión. |
| RF15 | Pasar a estado seguro ante desconexión, cancelación o reinicio. |
| RF16 | Reconectar automáticamente y sincronizar eventos pendientes. |
| RF17 | Evitar movimientos repetidos por comandos duplicados o vencidos. |
| RF18 | Calibrar sensores, revisar actuadores y guardar la versión de configuración. |

### 3. Actores y sistemas externos

El límite de contexto abarca el producto completo, incluyendo la aplicación, los servicios remotos y el dispensador. Por esa razón, Laravel, MySQL, Mosquitto, los sensores y los actuadores no se consideran actores externos. El proveedor de alojamiento y la infraestructura de red sí pertenecen al entorno.

| Actor o sistema | Tipo | Interacción principal |
|---|---|---|
| Persona cuidadora | Actor humano | Inicia sesión, configura mascota y porción, solicita controles y consulta resultados. |
| Chiquin y Tito | Actores físicos | Se aproximan al plato; el tag del collar permite identificar a la mascota. |
| Responsable de mantenimiento | Rol humano | Limpia, rellena, calibra y revisa el equipo. Puede ser la misma persona cuidadora. |
| Router Wi-Fi e Internet | Infraestructura externa | Transporta mensajes entre el ESP32 y los servicios remotos. |
| Proveedor del VPS | Servicio externo | Proporciona alojamiento y red para Laravel, Mosquitto y MySQL. |
| Servicio de hora | Servicio externo de apoyo | Permite sincronizar el reloj usado para validar la vigencia de los comandos. |

El backend autoriza las operaciones del usuario. La identificación por UID sirve para controlar el acceso de las mascotas dentro del prototipo. El corto alcance del RC522 exige probar posición y orientación del tag en el collar [3].

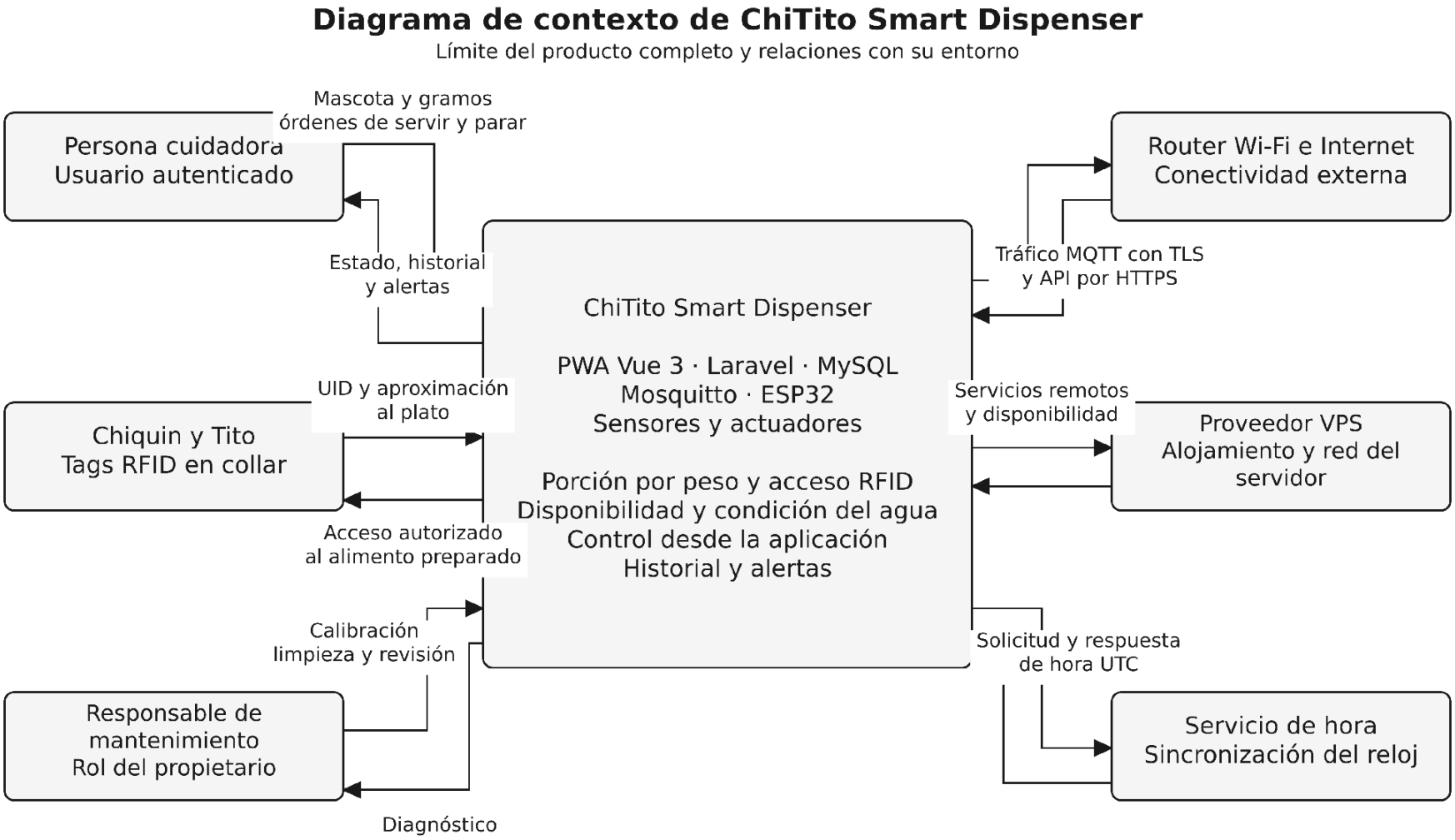



**Figura 1. Contexto del producto completo. Los bloques externos representan actores y servicios del entorno.**

**4. Casos de uso principales**

Se definen diez casos de uso. CU03 y CU05 se desarrollan por completo en la sección siguiente porque concentran el movimiento de actuadores y la respuesta ante fallas. CU04 se coordina con la región de alimento de la máquina de estados.

| ID | Caso de uso | Actor principal | Resultado esperado |
|---|---|---|---|
| CU01 | Iniciar sesión | Persona cuidadora | Obtener una sesión válida para los dispositivos permitidos. |
| CU02 | Configurar mascotas y porción | Persona cuidadora | Guardar nombre, UID activo, mascota seleccionada y cantidad dentro de los límites. |
| CU03 | Preparar porción de alimento | Persona cuidadora | Solicitar una dosificación y conocer el peso final y el resultado real. |
| CU04 | Acceder al plato por RFID | Chiquin o Tito | Abrir el acceso para el UID autorizado de una porción preparada. |
| CU05 | Controlar recirculación | Persona cuidadora | Activar o parar la bomba con validación y supervisión local del nivel. |
| CU06 | Consultar estado y condición | Persona cuidadora | Ver niveles, peso, TDS, turbidez, estados y antigüedad de las lecturas. |
| CU07 | Consultar historial y alertas | Persona cuidadora | Revisar porciones y eventos, y marcar una alerta como atendida. |
| CU08 | Recuperar conexión y sincronizar | Infraestructura de red | Restablecer comunicación y persistir eventos sin repetir movimientos. |
| CU09 | Realizar mantenimiento | Responsable de mantenimiento | Limpiar, rellenar, calibrar y probar salidas en modo supervisado. |

| ID | Caso de uso | Actor principal | Resultado esperado |
|---|---|---|---|
| CU10 | Detener una operación | Persona cuidadora | Solicitar cierre del dosificador y parada de bomba; pausar el acceso. |

### 4.1 Reglas comunes de interacción

- La PWA se comunica con Laravel mediante HTTPS. El backend aplica permisos y es el único componente de la aplicación que consulta MySQL.
- El ESP32 y Laravel intercambian JSON a través de Mosquitto con TLS y credenciales restringidas por dispositivo. La PWA no publica órdenes directamente en el broker.
- Toda orden tiene command_id y vigencia. Un nuevo clic o reintento HTTP conserva su clave de idempotencia mientras corresponda a la misma solicitud.
- Una porción pendiente bloquea otra dosificación hasta terminar, revisar los restos o cancelar de forma explícita. El agua se supervisa en una región independiente.
- La interfaz diferencia pendiente, aceptado, en ejecución y resultado terminal. Solo muestra porción confirmada cuando existe el evento final persistido.

## 5. Descripción completa de dos casos críticos

### 5.1 CU03 Preparar una porción para la mascota seleccionada

Propósito. Servir una masa configurable de alimento y conservar la autorización de acceso para Chiquin o Tito. La operación termina con una porción preparada o con un resultado de rechazo, cancelación o error. El acceso posterior corresponde a CU04.

Actores. La persona cuidadora inicia el caso; intervienen PWA, Laravel, MySQL, Mosquitto, ESP32, HX711, celda de carga, VL53L1X y el servo de dosificación. El servo de acceso permanece cerrado durante la preparación.

Disparador. El usuario selecciona una mascota, indica los gramos y presiona servir en la aplicación. Prioridad alta por el riesgo de dosificación excesiva o atribución incorrecta de la porción.

**Precondiciones**

- Usuario autenticado con permiso sobre el dispositivo; mascota activa y UID registrado.
- ESP32 con enlace MQTT y disponibilidad del backend vigentes, reloj sincronizado y configuración aplicada.
- Celda calibrada, peso estable, plato disponible y sin restos de otra porción. Depósito con alimento utilizable.
- Sin dosificación, mantenimiento o sesión pendiente incompatible. Límite de porción, tiempo máximo y tolerancia configurados.

**Flujo principal**

**1.** La PWA envía a Laravel dispositivo, mascota, cantidad en gramos y clave de idempotencia.

**2.** Laravel verifica sesión, permiso, asociación del UID, rango de cantidad y disponibilidad reciente del ESP32.

**3.** En una transacción, el backend guarda el comando con su vencimiento y una salida pendiente de publicación. MySQL confirma el COMMIT.

**4.** La PWA recibe HTTP 202 con command_id y estado pendiente. Un proceso del backend publica el comando MQTT sin retención.

**5.** El ESP32 comprueba destino, esquema, command_id, vigencia, versión de configuración, estado libre y condiciones físicas locales.

**6.** Antes de abrir la salida, registra el inicio del comando en memoria no volátil. Mantiene cerrado el acceso al plato y toma el peso inicial estable.

**7.** El servo 1 abre la compuerta de alimento. El ESP32 calcula la masa agregada a partir de lecturas filtradas del HX711.

**8.** Al acercarse al objetivo reduce la apertura según la calibración mecánica. Cierra en el umbral configurado para compensar el alimento que sigue cayendo.

**9.** Con la salida cerrada, espera estabilización y obtiene el peso final. La masa servida es peso final menos peso inicial; verifica tolerancia y ausencia de error.

**10.** Guarda el resultado, la masa servida y una sesión de porción asociada al UID autorizado. Entra en espera RFID sin volver a dosificar.

**11.** Publica el evento final con event_id y command_id. Laravel guarda el resultado y la porción en MySQL de manera idempotente.

**12.** La PWA consulta el comando y muestra porción preparada, gramos reales y mascota autorizada. Cuando el tag correcto se aproxima, CU04 administra el acceso.

**Flujos alternos y excepciones**

| Condición | Respuesta prevista |
|---|---|
| Cantidad o permisos inválidos | Laravel rechaza la solicitud y no publica ninguna orden de movimiento. |
| Comando repetido | ESP32 devuelve el estado o resultado guardado; no vuelve a abrir el dosificador. |
| Plato con restos o recurso ocupado | Rechaza la preparación e informa revisión del plato o sesión pendiente. |
| Comando vencido o reloj no válido | No inicia la operación. Requiere una nueva solicitud una vez recuperadas las condiciones. |

| Condición | Respuesta prevista |
|---|---|
| HX711 inválido, peso sin crecimiento o tiempo máximo | Cierra la salida, bloquea el acceso y registra error o porción parcial si el peso es confiable. |
| Porción final fuera de tolerancia | Conserva la masa medida y registra fuera de tolerancia. No corrige agregando otra dosis automáticamente. |
| Cancelación o pérdida de enlace durante la caída | Cierra localmente el dosificador. Registra cancelado o interrumpido y exige revisión antes de otra porción. |
| Pérdida de enlace después de preparar la porción | Pausa el acceso y conserva peso, UID y sesión. Revalida la porción desde la app sin repetir la dosificación. |
| Reinicio durante la ejecución | Arranca seguro y registra ejecución interrumpida. No reconstruye un resultado exitoso ni repite el comando. |
| Resultado enviado pero no persistido | Conserva el evento en cola y lo reenvía con el mismo event_id hasta recibir ACK de aplicación. |

**Postcondiciones y criterios de aceptación**

En éxito, existe un único resultado confirmado para command_id, con masa real medida, mascota y UID autorizado. El dosificador queda cerrado y el sistema espera RFID. En error, los actuadores se ordenan a estado seguro; un peso no confiable se guarda como nulo, acompañado de su motivo, y nunca se reemplaza por los gramos solicitados.

La aceptación exige impedir una segunda ejecución del mismo command_id, cerrar ante la condición de parada, conservar la diferencia entre objetivo y masa real y mantener cerrado el acceso para un UID distinto. La tolerancia y el tiempo de cierre se validarán con el prototipo. Figura 2 describe el control local; Figura 4 cubre el transporte y la persistencia.

### 5.2 CU05 Controlar la recirculación con protección por nivel

Propósito. Activar o detener la bomba de agua desde la aplicación y apagarla localmente cuando el nivel sea insuficiente o su lectura no sea válida. El caso controla recirculación, sin asignar mililitros ni consumo de agua por mascota.

Actores. Persona cuidadora como actor principal; PWA, Laravel, MySQL, Mosquitto, ESP32, A02YYUW, AO3400A y bomba de 5 V como componentes participantes. SEN0244 y SEN0189 aportan avisos de condición del agua mediante CU06.

Disparador. El usuario solicita activar o detener recirculación. Una condición de nivel bajo o inválido también dispara la parada local. Prioridad alta por la protección de la bomba.

#### Precondiciones

- Para activar: sesión autorizada, enlace disponible, comando vigente y configuración de nivel aplicada.
- Depósito instalado, agua sobre el umbral de arranque y lectura A02YYUW reciente y válida.
- Bomba, MOSFET y alimentación revisados, sin mantenimiento activo ni bloqueo pendiente.
- Para una parada de protección no se requiere conectividad ni respuesta del servidor.

#### Flujo principal de activación y supervisión

**1.** La PWA solicita activar la bomba. Laravel valida usuario, dispositivo, disponibilidad y clave de idempotencia.

**2.** Laravel guarda el comando y la publicación pendiente en MySQL. Publica la orden vigente por MQTT y entrega a la PWA el identificador de seguimiento.

**3.** El ESP32 valida command_id y vigencia, y obtiene lecturas filtradas del A02YYUW.

**4.** Convierte distancia a nivel usando los puntos de calibración. Si el nivel supera el umbral de arranque y no hay bloqueo, activa la salida AO3400A.

**5.** Registra y publica bomba activada como estado de salida ordenada. La PWA lo muestra una vez persistido el evento.

**6.** Durante la activación, el ESP32 continúa verificando nivel, vigencia de disponibilidad del backend y órdenes de parada.

**7.** Si recibe una orden de parada, detecta nivel bajo o inválido, o pierde enlace, desactiva AO3400A localmente sin esperar a Laravel.

**8.** Guarda la causa de parada; genera alerta cuando corresponde y envía el evento. Si no hay conexión, lo conserva para sincronización.

**9.** El backend persiste la nueva condición y la aplicación actualiza el estado. La recuperación del nivel no reactiva la bomba automáticamente.

**Flujos alternos y excepciones**

| Condición | Respuesta prevista |
|---|---|
| Nivel inicial insuficiente o inválido | Mantiene bomba apagada y rechaza encendido con la causa correspondiente. |
| Lectura oscila cerca del mínimo | Utiliza filtrado e histéresis. El umbral de arranque es superior al de parada. |
| Parada recibida cuando ya estaba apagada | Confirma apagada; no produce una acción adicional peligrosa. |
| TDS o turbidez fuera del umbral de aviso | Genera aviso de revisión o cambio de agua. No declara agua potable o contaminada con certeza. |
| Falla Wi-Fi, MQTT o backend | Apaga la bomba al detectar caída del enlace o vencimiento del lease del backend. |
| Nivel recuperado o reconexión exitosa | Permanece apagada hasta una solicitud nueva del usuario. |
| Salida activada sin circulación observable | No puede detectarlo automáticamente con el hardware actual. El responsable revisa bomba, manguera y alimentación. |

**Postcondiciones y criterios de aceptación**

Al activar correctamente, el firmware confirma la salida de control encendida y mantiene supervisión por nivel. Al parar o detectar falla, confirma la salida apagada y conserva un

evento con su causa. El estado informado no demuestra flujo físico porque no hay caudalímetro en el alcance.

La aceptación exige bloquear encendido por nivel inválido o bajo, aplicar la parada local antes de cualquier comunicación remota y mantener la bomba apagada después de una reconexión o recuperación del nivel. Figura 3 detalla esta actividad; Figura 7 representa sus estados.

### 6. Diagramas de actividad

Las dos actividades se concentran en las decisiones del ESP32. El transporte frontend, backend y broker se muestra en la secuencia integral, para evitar repetirlo dentro de cada actividad. Los rombos contienen condiciones de decisión; los rectángulos contienen acciones y los círculos marcan inicio o final del flujo.

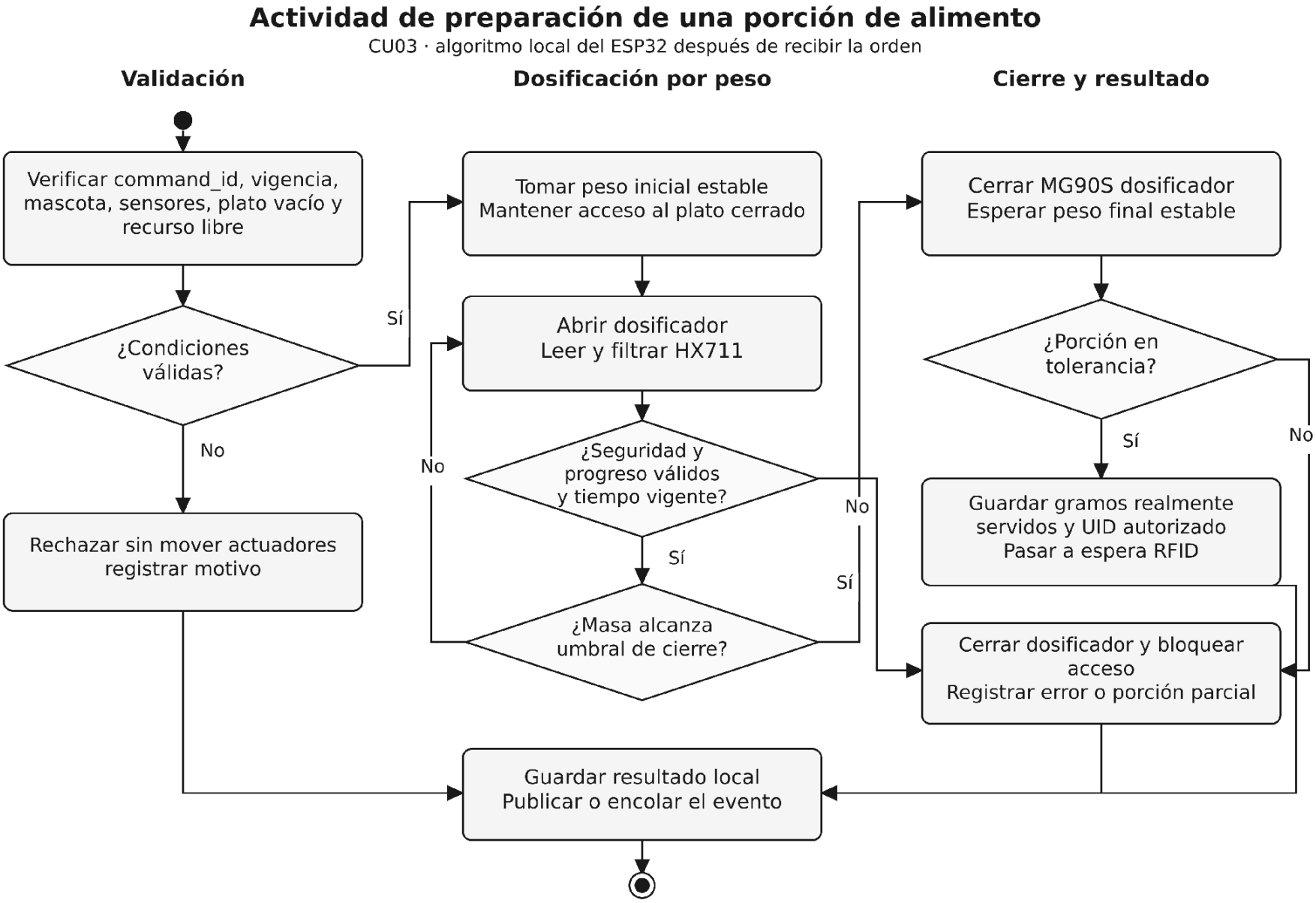



**Figura 2. Actividad local de CU03. El acceso permanece cerrado hasta que la porción se verifica y se asocia al UID autorizado.**

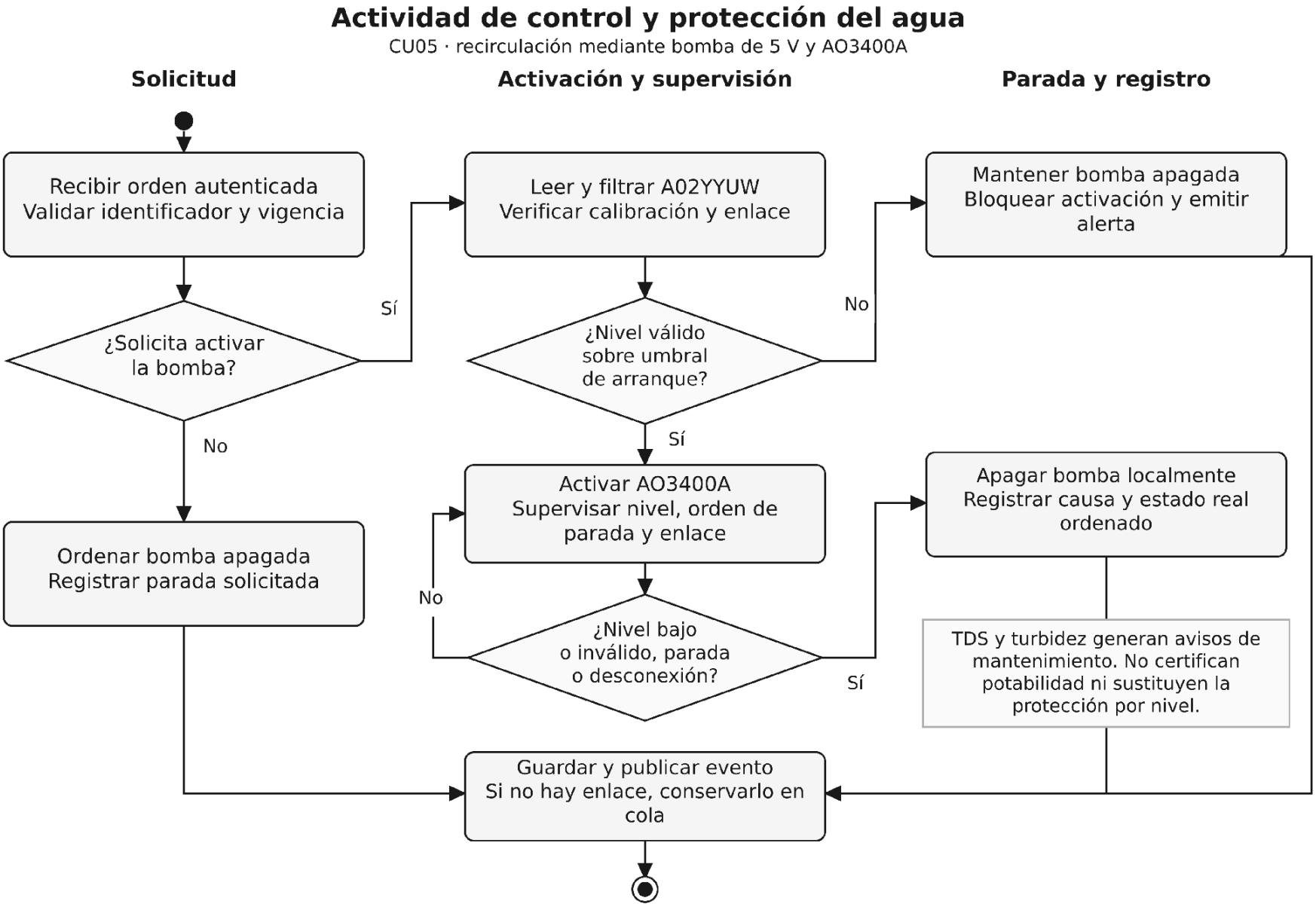



**Figura 3. Actividad local de CU05. Una lectura de nivel inválida también produce parada y bloqueo de encendido.**

## 7. Secuencia integral y contrato de comunicación

La Figura 4 es una sola secuencia, distribuida en dos páginas consecutivas para conservar la legibilidad. Primero muestra sensor, firmware, backend, MySQL y frontend; después incluye el retorno de una solicitud de control hasta el actuador y la confirmación del resultado. Los sensores y actuadores se representan dentro del ESP32 mediante la referencia al algoritmo de la Figura 2.

Los mensajes 1 a 11 cubren adquisición, persistencia y consulta. Los mensajes 12 a 18 cubren la solicitud desde la app, el COMMIT previo a publicar y la validación local. Los mensajes 19 a 28 cubren eventos de estado, resultado final, confirmación de persistencia y consulta del frontend. El ciclo de eventos también admite un rechazo del comando; el rechazo no implica movimiento del servo.

### 7.1 Semántica de comandos y resultados

La propuesta utiliza MQTT con QoS 1 para comandos y resultados, y retain false para órdenes de movimiento. QoS 1 puede entregar duplicados, por lo que la idempotencia se implementa en la aplicación y el ESP32. Un PUBACK del broker no demuestra que MySQL guardó el resultado ni que ocurrió una dosificación [2].

Laravel guarda comando y salida pendiente en la misma transacción. Si el proceso que publica a MQTT falla, puede recuperar esa salida sin crear otra solicitud. El ESP32 registra el inicio antes de actuar. El backend confirma event_id únicamente después de COMMIT; el dispositivo retira ese evento de la cola local tras recibir el ACK de aplicación.

Un comando puede pasar por pendiente, aceptado, en_ejecucion y confirmado, o terminar en rechazado, vencido, cancelado o error. Si vence el seguimiento de una orden que pudo comenzar y no hay resultado terminal, la interfaz muestra sin_confirmacion. Ese estado se reconcilia con el ESP32 antes de permitir un nuevo intento que agregue alimento.

### 7.2 Tópicos previstos

| Tópico | Publica y recibe | Contenido |
|---|---|---|
| chitito/{id}/telemetry | ESP32 → Laravel | Lecturas, calidad, estados y fechas. |

| Tópico | Publica y recibe | Contenido |
|---|---|---|
| chitito/{id}/events | ESP32 → Laravel | Estado de comando, porción, acceso y alertas. |
| chitito/{id}/commands | Laravel → ESP32 | SERVIR, BOMBA_ON, BOMBA_OFF, PARAR y REVALIDAR. |
| chitito/{id}/config | Laravel → ESP32 | Configuración versionada; solo datos, sin ejecución de movimiento. |
| chitito/{id}/persisted | Laravel → ESP32 | event_id guardado después de COMMIT. |
| chitito/{id}/availability | ESP32 → Laravel | Disponibilidad y Last Will del dispositivo. |
| chitito/{id}/backend_status | Laravel → ESP32 | Disponibilidad del consumidor y lease de control, sin retención. |

Las ACL permiten al ESP32 publicar únicamente sus datos y suscribirse a sus órdenes y confirmaciones. El backend tiene permisos de publicación y consumo acordes con su función. Una configuración retenida puede conservar la última versión; nunca debe convertirse en una orden de abrir o activar al reconectar.

### 7.3 Vigencia y disponibilidad

Como valores iniciales de diseño se propone heartbeat cada 10 s, lease del backend de 30 s y vigencia de comandos de movimiento de 30 s. Son parámetros de supervisión, no mediciones del prototipo. El backend renueva el lease solo si su consumidor y MySQL están operativos. Un mensaje repetido o atrasado no renueva la disponibilidad; se verifica su sesión y secuencia.

La pérdida detectada de Wi-Fi o MQTT inicia el estado seguro inmediatamente. Si Mosquitto sigue conectado pero Laravel deja de funcionar, el lease evita considerar operativo todo el sistema. El frontend informa última lectura y última disponibilidad; si está desconectado, muestra datos históricos con fecha y bloquea controles nuevos.

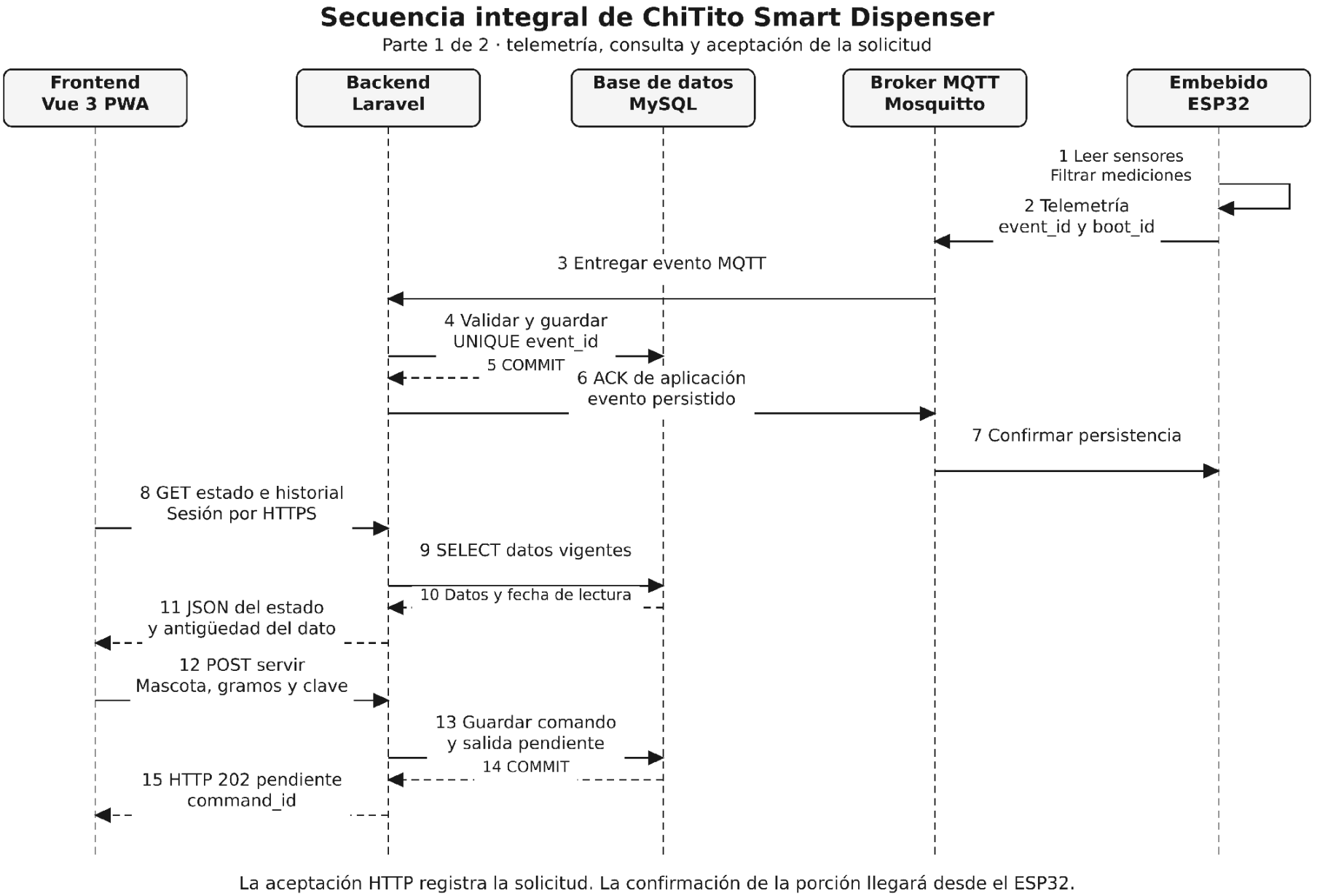
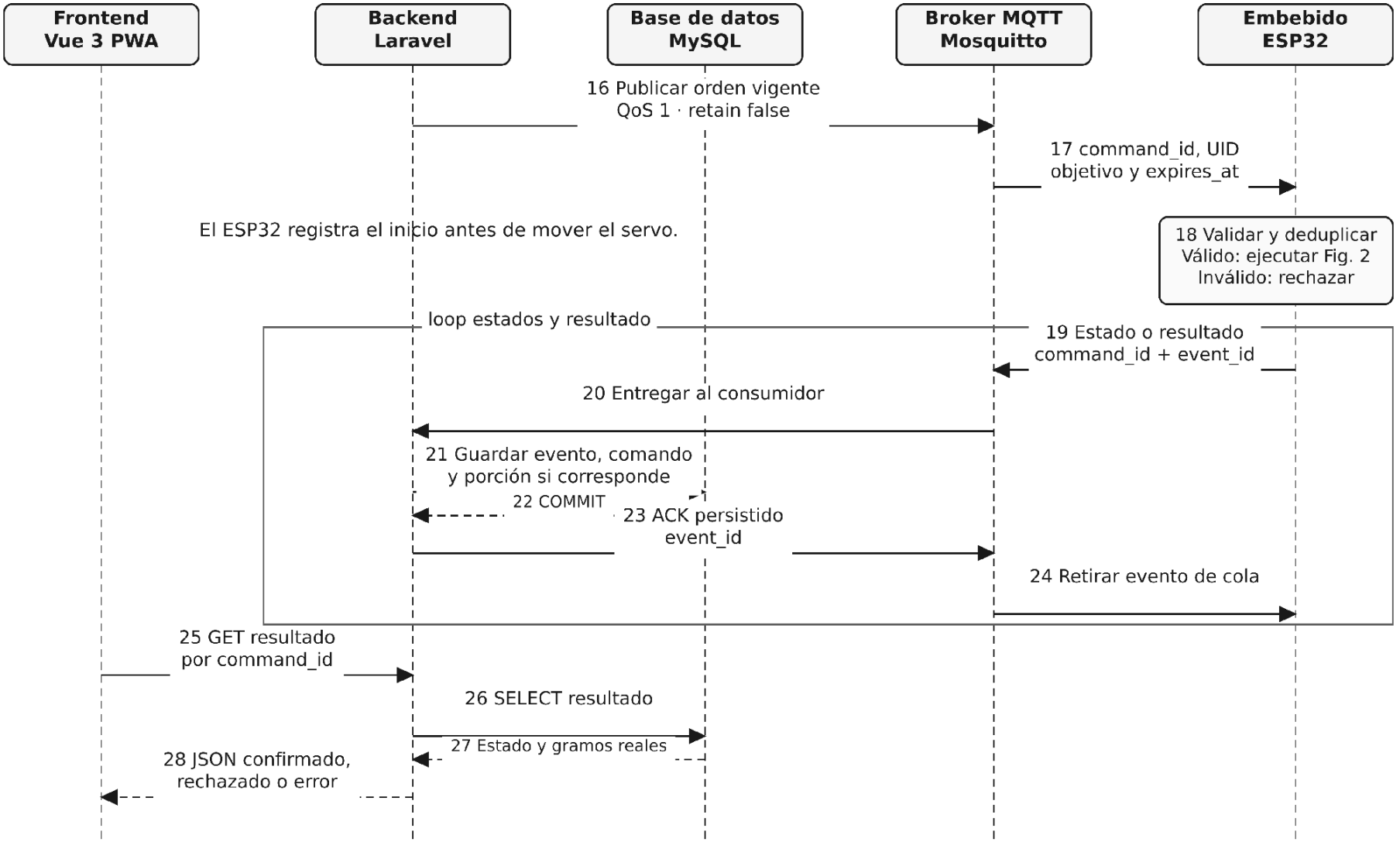



**Figura 4. Secuencia integral, parte 1 de 2. Adquisición y consulta de datos, y registro de una solicitud de alimento.**

# Secuencia integral de ChiTito Smart Dispenser

Parte 2 de 2 · control local, resultado persistido y actualización del frontend




Si se pierde el enlace: parada segura y cola local. La reconexión sincroniza eventos y no repite movimientos.

**Figura 4. Continuación, parte 2 de 2. Publicación del comando, ejecución local y resultado confirmado después de persistirlo.**

### 8. Máquina de estados compuesta del sistema embebido

El comportamiento sí depende de estados identificables: dosificar, verificar peso, esperar RFID, permitir acceso, bloquear por nivel y recuperar conexión. Se modela una máquina compuesta con tres regiones concurrentes. El estado completo es la combinación de conectividad C, alimento A y agua W, por ejemplo C3, A5 y W0: en línea, esperando RFID y bomba apagada.

La región de conectividad supervisa a las otras dos. C3 habilita órdenes nuevas; C0, C1, C2 y C4 bloquean dosificación y encendido. La región de alimento administra la porción y su acceso; la de agua mantiene la protección de nivel aun cuando se esté preparando alimento.

#### 8.1 Reglas de prioridad y recuperación

**1.** Al iniciar o reiniciar: salida de bomba apagada, dosificador ordenado a cerrar y acceso sin nueva apertura. Se recupera el registro local para identificar operaciones interrumpidas.

**2.** Un error de sensor crítico, cancelación o pérdida de enlace interrumpe la dosificación. El nivel bajo o inválido apaga la bomba sin depender de otra región.

**3.** Si el acceso estaba abierto, la región de alimento realiza cierre gradual antes de quedar bloqueada. La temporización RFID ayuda a reducir cierres por lecturas intermitentes, pero no confirma que una cabeza esté fuera del mecanismo.

**4.** Una orden duplicada se responde usando el registro existente. No produce transición de dosificación ni reinicio de una acción terminal.

**5.** La sincronización conserva gramos, sesión de mascota, UID y resultados pendientes. Una porción preparada se revalida desde la app; no se vuelve a llenar.

**6.** Para otra dosificación se verifica plato libre. Los restos requieren cerrar la sesión o revisar el plato; el sistema no los oculta mediante una tara automática.

#### 8.2 Eventos y condiciones de guarda

| Evento o guarda | Significado |
|---|---|
| comando_vigente | Destino correcto, command_id nuevo, esquema válido y expires_at aún válido. |

| Evento o guarda | Significado |
|---|---|
| enlace_ok | Wi-Fi y MQTT activos, backend disponible y lease actual vigente. |
| peso_estable | Lecturas válidas con dispersión bajo el umbral de estabilidad configurado. |
| peso_de_cierre | Masa agregada alcanza objetivo menos compensación de alimento en caída. |
| porcion_ok | Masa final confiable dentro de la tolerancia configurada respecto del objetivo. |
| uid_autorizado | Tag leído coincide con el UID guardado para la sesión de alimento actual. |
| uid_ausente_estable | No se detecta el UID durante el intervalo configurado; no equivale a presencia física descartada. |
| nivel_ok y nivel_bajo | Lectura reciente y calibrada; comparación con umbrales separados de arranque y parada. |
| fallo o timeout | Sensor inválido, ausencia de progreso, duración máxima o condición local incompatible. |

### 8.3 Comportamiento del acceso RFID

En A5, el ESP32 conserva una porción lista y un único UID autorizado. Un tag distinto mantiene el acceso cerrado y genera un evento de acceso denegado con limitación de frecuencia. El tag correcto abre el servo 2 y pasa a A6. La pérdida temporal de lectura no provoca cierre brusco; se utiliza un intervalo de ausencia y un movimiento gradual.

Después del cierre, si quedan restos y la sesión sigue vigente, regresa a A5 para permitir a la misma mascota continuar. Si el plato queda vacío o el usuario termina la sesión, vuelve a A0. Un cambio de mascota se aplica a la siguiente porción, después de cerrar o revisar la sesión actual.

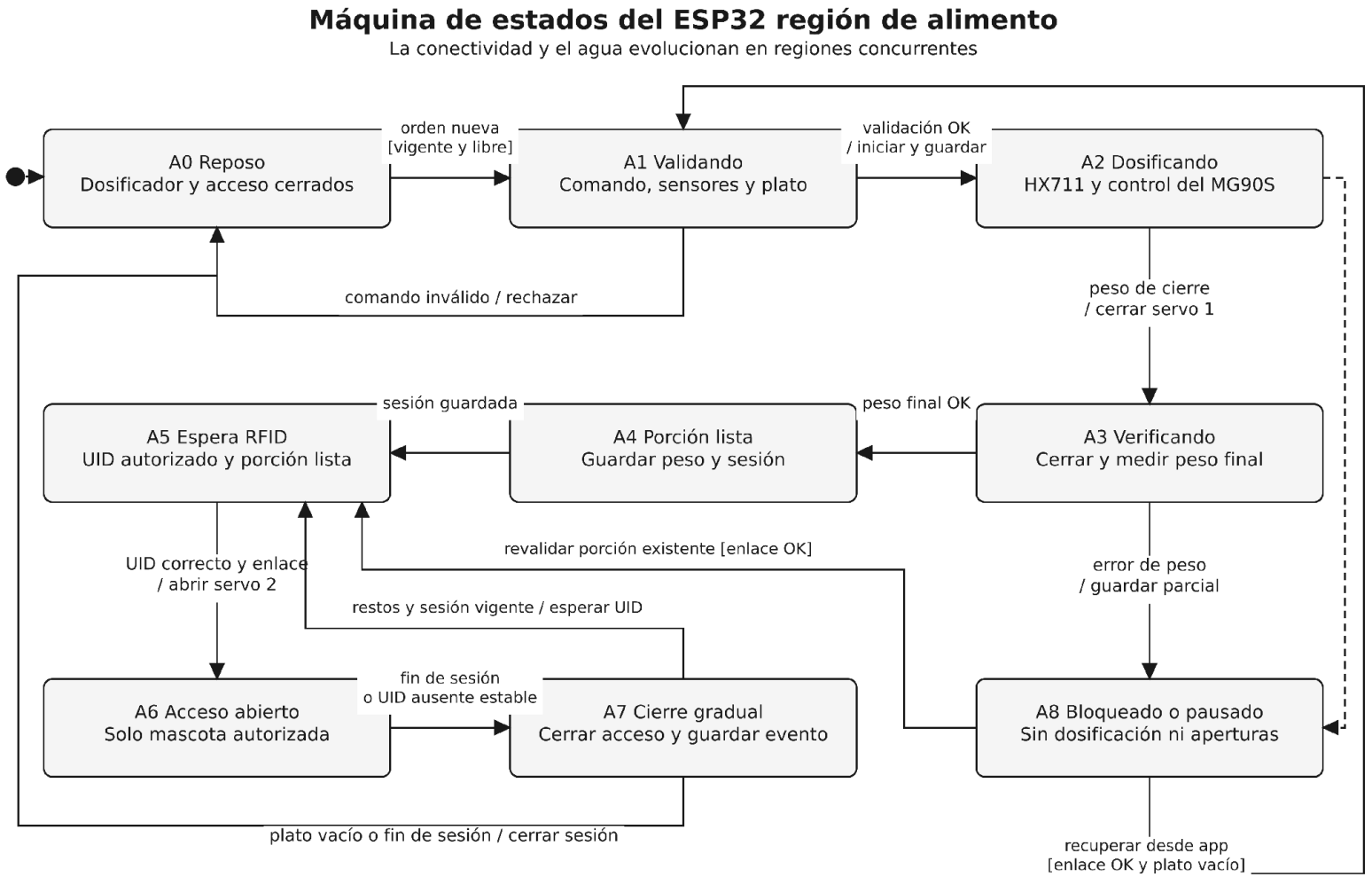



**Figura 5. Región de alimento y acceso RFID. Las transiciones de recuperación separan revalidar una porción de preparar otra.**

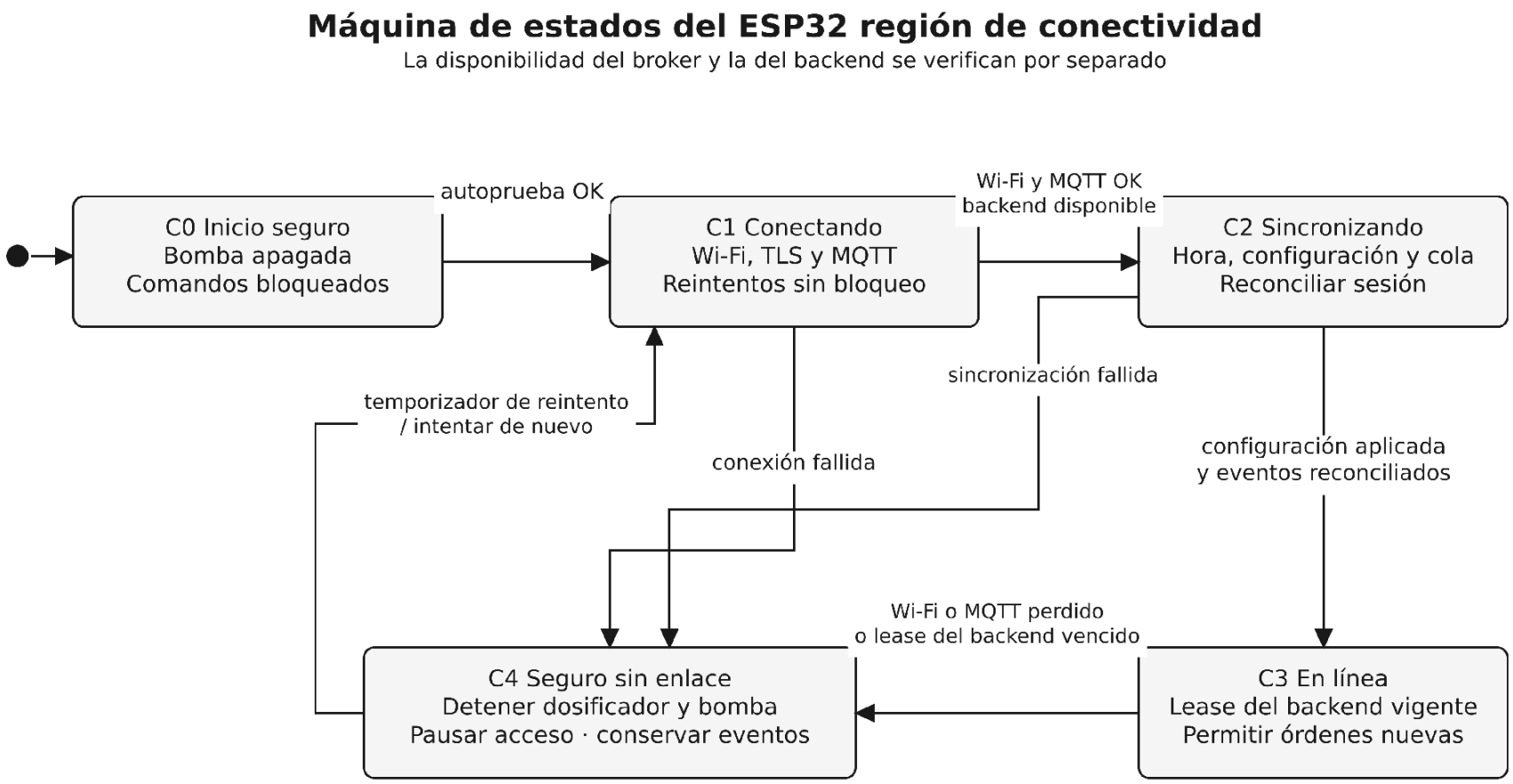



**Figura 6. Región de conectividad. El lease verifica al backend además del estado de Wi-Fi y MQTT.**

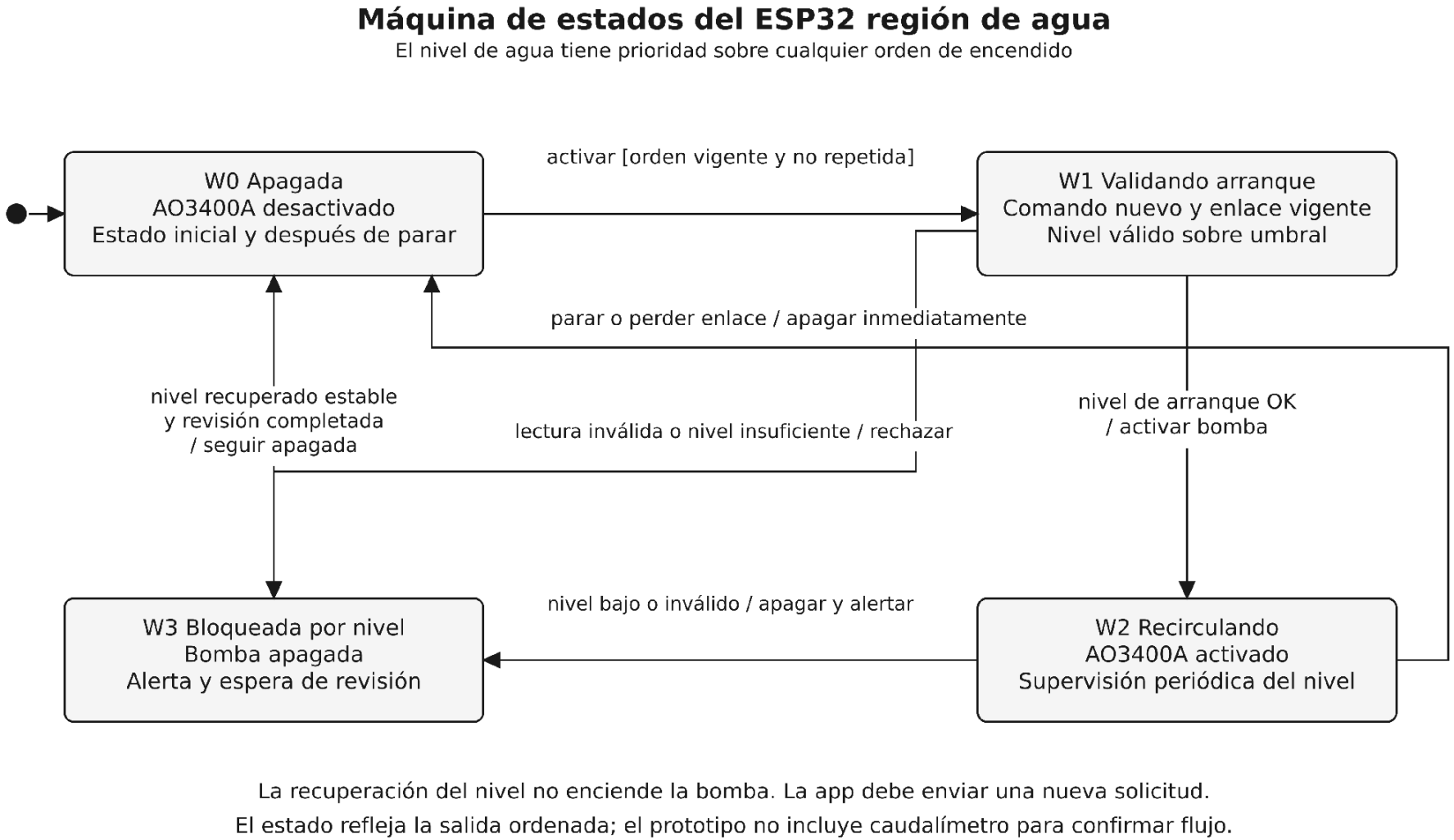



**Figura 7. Región de agua. La bomba permanece apagada hasta una orden nueva después de recuperar nivel o enlace.**

### 9. Diccionario preliminar de los datos principales

Los tipos representan una propuesta de esquema para MySQL y mensajes JSON. Los identificadores del usuario y las mascotas son claves internas; los identificadores de comandos y eventos se conservan al reintentar. Todas las fechas se almacenan en UTC y el frontend las presenta en la zona horaria del usuario.

#### 9.1 Entidades y relaciones

| Entidad | Relación y responsabilidad |
|---|---|
| usuarios | Propietario autenticado; se relaciona con mascotas y dispositivos permitidos. |
| mascotas | Pertenece a un usuario; contiene nombre y UID activo único. |
| dispositivos | Identifica al ESP32 y su propietario; conserva última disponibilidad y versión aplicada. |
| configuraciones | Versiones por dispositivo con límites, calibración, umbrales y reglas de sesión. |
| comandos | Pertenece a dispositivo y usuario; puede referir mascota y genera como máximo una porción. |
| porciones | Relaciona comando y mascota; registra objetivo, masa real y resultado de preparación. |
| telemetrias | Serie de lecturas por dispositivo, con fecha y calidad de medición. |
| eventos y alertas | Eventos deduplicados por event_id; alertas vinculadas a dispositivo y causa. |
| salidas_pendientes | Mensajes que el backend debe publicar; se guardan junto con la operación de negocio. |

### 9.2 Identificación y configuración

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| usuario_id | BIGINT | PK de usuarios. Se usa para autorización; no reemplaza las credenciales MQTT. |
| password_hash | VARCHAR(255) | Hash generado por backend; nunca se envía a ESP32 ni se devuelve a la PWA. |
| mascota_id y nombre | BIGINT y VARCHAR(60) | FK del propietario; nombres iniciales Chiquin y Tito. |
| rfid_uid | VARCHAR(20) | Hexadecimal normalizado; longitud de 8, 14 o 20 caracteres según tag. Único entre tags activos. |
| device_id | VARCHAR(36) | Identificador estable del equipo; destino de tópicos y FK de datos. |
| config_version | INT UNSIGNED | Aumenta al cambiar calibración o límites; el ESP32 confirma la versión aplicada. |
| porcion_max_g y tolerancia_g | DECIMAL en g | Límite mecánico calibrado y banda permitida. No son recomendaciones de alimentación. |
| tare_g y factor_hx711 | DECIMAL | Tara del plato vacío y conversión calibrada. Se cambian en mantenimiento, no en cada porción. |
| cierre_anticipado_g | DECIMAL en g | Compensación experimental del alimento que sigue cayendo al cerrar. |
| distancias_vacio_lleno_mm | DECIMAL en mm | Puntos de referencia separados para depósito de alimento y agua. |

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| nivel_parada_pct y nivel_arranque_pct | DECIMAL en % | 0 a 100; arranque mayor que parada para dar histéresis. |
| timeout_dosis_ms y timeout_progreso_ms | INT UNSIGNED en ms | Límites locales de duración y de peso sin crecimiento, medidos con reloj monotónico. |
| sesion_rfid_ms y ausencia_rfid_ms | INT UNSIGNED en ms | Duración de autorización y filtro de ausencia; no son detección física de salida. |
| umbral_tds y umbral_turbidez | DECIMAL | Umbrales de aviso relativos a calibración y referencia del agua. No son límites sanitarios. |

### 9.3 Comandos y porciones

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| command_id | UUID en CHAR(36) | Único y estable; conserva su valor en MQTT y reintentos. |
| idempotency_key | VARCHAR(64) | Clave de solicitud PWA; única por usuario y operación dentro del periodo conservado. |
| accion | ENUM o VARCHAR(24) | SERVIR, BOMBA_ON, BOMBA_OFF, PARAR, REVALIDAR o mantenimiento autorizado. |
| issued_at y expires_at | DATETIME(3) UTC | Generados por backend. Una orden de movimiento vencida no se ejecuta. |

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| estado_comando | VARCHAR(24) | pendiente, aceptado, en_ejecucion, confirmado, rechazado, vencido, cancelado, error o sin_confirmacion. |
| error_code | VARCHAR(40) nullable | Causa estable para UI y auditoría; no contiene secretos ni mensajes arbitrarios ejecutables. |
| porcion_id | BIGINT | PK de porciones; command_id es FK con restricción UNIQUE. |
| objetivo_g | DECIMAL(8,2) en g | Mayor que cero y dentro del máximo configurado; lo solicita el usuario. |
| peso_inicial_g y peso_final_g | DECIMAL(8,2) en g | Mediciones estables HX711; nulas si no fue posible una lectura válida. |
| servido_g | DECIMAL(8,2) en g nullable | Peso final menos inicial. Se conserva también si está fuera de tolerancia; nulo si no es confiable. |
| resultado_porcion | VARCHAR(24) | preparada, parcial, fuera_tolerancia, cancelada o interrumpida; no equivale a consumida. |
| sesion_id y uid_autorizado | UUID y VARCHAR(20) | Asocia el acceso a una única porción y mascota. Persistente mientras haya sesión válida. |
| preparada_at y finalizada_at | DATETIME(3) nullable | Fechas del evento; el cierre de sesión se distingue del fin de la dosificación. |

### 9.4 Telemetría y eventos

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| event_id | UUID en CHAR(36) | UNIQUE en backend; mismo identificador al reenviar desde la cola del ESP32. |
| boot_id y seq | UUID e INT UNSIGNED | Identifican el arranque y orden local. No convierten un evento antiguo en actual. |
| measured_at y received_at | DATETIME(3) UTC | Momento de adquisición y recepción. Si no había hora válida, measured_at queda nulo. |
| uptime_ms | BIGINT en ms | Tiempo monotónico desde el arranque; permite ordenar eventos aun sin hora UTC. |
| peso_plato_g | DECIMAL(8,2) en g | Peso observado del plato; no demuestra alimento ingerido. |
| alimento_pct y agua_pct | DECIMAL(5,2) en % | Estimaciones 0 a 100, derivadas de calibración. Lectura inválida produce valor nulo, no cero. |
| tds_ppm | DECIMAL(7,2) nullable | Estimación SEN0244. Se guarda referencia de calibración y temperatura asumida si no hay sensor térmico. |
| turbidez_adc y turbidez_v | INT y DECIMAL en V | Se conserva señal original y voltaje reconstruido tras el acondicionamiento. No se expresa NTU sin calibración válida. |

| Dato | Tipo o unidad | Regla y origen |
|---|---|---|
| calidad_lectura | JSON o VARCHAR(24) | Por sensor: válida, inestable, fuera_rango, sin_dato o calibración_pendiente. |
| estado_c, estado_a y estado_w | VARCHAR(24) | Estados de las tres regiones del firmware según Figuras 5 a 7. |
| salida_bomba y servo_ordenado | BOOL y DECIMAL en grados | Salida lógica y posición solicitada. No constituyen realimentación física. |
| backend_lease_seq y lease_expires | INT y DATETIME(3) | Lease nuevo de una sesión activa del backend; los mensajes viejos no renuevan control. |
| last_seen_at y online | DATETIME(3) y BOOL | Disponibilidad calculada con heartbeat y Last Will; la UI muestra antigüedad. |
| alerta_id, tipo y severidad | BIGINT y VARCHAR | Nivel bajo, sensor, enlace, acceso denegado u operación incompleta; severidad informativa, aviso o crítica. |
| alerta_activa y atendida_at | BOOL y DATETIME(3) nullable | Atender una alerta no elimina una falla todavía presente. |
| evento_payload | JSON versionado | Valores del resultado y schema_version; se valida tamaño, campos y tipos antes de persistir. |

El A02YYUW entrega distancia por UART; los porcentajes dependen de la geometría del depósito [4]. TDS requiere calibración y una referencia de temperatura; no se incorpora un DS18B20 que no esté en los componentes seleccionados. La lectura analógica del SEN0189 requiere acondicionamiento para la entrada ADC del ESP32 [5], [6].

### 9.5 Datos locales persistentes del ESP32

La memoria no volátil conserva versión de configuración, calibración, registros de comandos recientes, estado de una operación interrumpida, sesión de porción y cola de eventos aún no confirmados. El command_id se marca antes del movimiento para impedir su repetición después de un reinicio. Si hubo corte eléctrico, la operación se conserva como interrumpida y exige revisión, aunque no pueda conocerse cuánto alimento cayó.

La cola se limita a la capacidad reservada. Los resultados terminales y las fallas tienen prioridad sobre telemetría repetitiva. Si no queda espacio para registrar de forma segura una nueva dosificación, se bloquea esa dosificación y se emite un aviso. No se descarta un resultado terminal sin ACK de persistencia para sustituirlo por una lectura reciente.

## 10. Manejo previsto de errores y desconexiones

Cada error tiene una respuesta local, un registro y un efecto visible. El frontend conserva el último dato con fecha, pero bloquea acciones cuando no puede confirmar disponibilidad. Los reintentos del dispositivo usan temporizadores sin bloquear el control de peso o nivel; se propone espera creciente de 1, 2, 4, 8, 16 y hasta 30 s, con variación pequeña para evitar reconexiones simultáneas.

| Situación y detección | Respuesta del sistema | Registro y recuperación |
|---|---|---|
| Caída Wi-Fi o sesión MQTT | C4; cerrar dosificador, apagar bomba y pausar acceso. Continuar supervisión local. | Evento local y reconexión automática; al recuperar, sincronizar sin repetir movimientos. |
| Backend caído con broker activo | Vence lease sin heartbeat nuevo; aplicar el mismo estado seguro. | PWA informa indisponibilidad. C2 requiere backend y persistencia sanos antes de C3. |
| MySQL no disponible | Backend no confirma persistencia ni acepta nuevos movimientos. Deja de renovar disponibilidad de control. | ESP32 conserva eventos; el consumidor reintenta y confirma solo después de COMMIT. |

| Situación y detección | Respuesta del sistema | Registro y recuperación |
|---|---|---|
| Publicación MQTT fallida | Comando queda pendiente en salida transaccional; no se presume ejecutado. | Reintentar mismo command_id dentro de su vigencia. Si pudo iniciarse, reconciliar antes de repetir. |
| Duplicado, destino incorrecto o JSON inválido | Validar y rechazar. Un duplicado conocido recibe su resultado previo. | Guardar causa o contador de rechazo; nunca abrir por un mensaje mal formado. |
| Orden vencida o reloj no sincronizado | Bloquear encendido y dosificación. Las protecciones y la parada local siguen activas. | Sincronizar hora y solicitar orden nueva. No ampliar expires_at de un comando existente. |
| Corte eléctrico o reinicio | Arranque seguro; recuperar el diario y marcar ejecución incompleta. | Inspeccionar plato y mecanismo; el registro previo evita repetir la acción. |
| HX711 inválido o plato presionado | Cerrar dosificador y bloquear acceso; lectura no confiable se conserva como nula. | Revisar montaje, estabilidad y tara en CU09; no inventar gramos servidos. |
| Alimento no cae o depósito vacío | Detectar falta de progreso o nivel insuficiente; cerrar al timeout. | Alerta y porción parcial si es medible; usuario rellena o revisa la compuerta. |
| Exceso o defecto de porción | Guardar peso real y estado fuera de tolerancia. No agregar una segunda dosis automática. | Revisión de cierre anticipado, caída residual y calibración del HX711. |
| Agua baja o A02YYUW inválido | Apagar AO3400A y bloquear arranque. Aplicar histéresis y validación de lecturas. | Rellenar o corregir sensor; después se necesita una orden nueva para activar. |

| Situación y detección | Respuesta del sistema | Registro y recuperación |
|---|---|---|
| TDS o turbidez anormal o sensor sucio | Marcar calidad de lectura y emitir aviso de mantenimiento. | Limpiar y comparar con referencia; no declarar potabilidad ni cambio automático. |
| UID distinto o lectura intermitente | UID distinto no abre. Una ausencia breve se filtra antes de ordenar cierre gradual. | Evento de acceso con limitación de frecuencia; revisar ubicación del collar y lector. |
| Servo atascado o bomba sin flujo | No hay realimentación dedicada; el estado de salida no se presenta como comprobación mecánica. | Parar ante indicios medibles y solicitar revisión. Diagnóstico supervisado en CU09. |
| PWA sin Internet o sesión vencida | Mostrar datos con su fecha y deshabilitar comandos. No encolar órdenes de movimiento en la PWA. | Reconectar o iniciar sesión; consultar el resultado pendiente antes de crear otro comando. |
| Cola local llena o escritura fallida | Conservar resultados importantes y bloquear dosificación nueva si no se puede registrar. | Alerta de persistencia y diagnóstico. No borrar resultados sin confirmación del backend. |

**11. Matriz entre requisitos y casos de uso**

La marca X indica que el caso de uso implementa o verifica el requisito. La matriz se divide en dos partes consecutivas por legibilidad. CU01 a CU10 conservan los nombres de la sección 4; CU03 y CU05 contienen las especificaciones críticas completas.

**Matriz de trazabilidad requisitos 1 a 9**

| Requisito | CU01 | CU02 | CU03 | CU04 | CU05 | CU06 | CU07 | CU08 | CU09 | CU10 |
|---|---|---|---|---|---|---|---|---|---|---|
| RF01 Autenticación | X | X | X | | X | X | X | | X | X |
| RF02 Mascotas y UID | | X | | X | | | | | | |
| RF03 Selección y gramos | | X | X | | | | | | | |
| RF04 Control remoto | | | X | | X | | | | | X |
| RF05 Dosificación por peso | | | X | | | | | | X | |
| RF06 Registro de masa real | | | X | | | | X | | | |
| RF07 Acceso autorizado | | | X | X | | | | | | |
| RF08 Nivel de alimento | | | | | | X | | | X | |
| RF09 Agua y protección | | | | | X | X | | | X | |

CU01 sesión · CU02 configuración · CU03 alimento · CU04 RFID · CU05 agua · CU06 estado · CU07 historial · CU08 conexión · CU09 mantenimiento · CU10 parada.

**Matriz de trazabilidad requisitos 10 a 18**

| Requisito | CU01 | CU02 | CU03 | CU04 | CU05 | CU06 | CU07 | CU08 | CU09 | CU10 |
|---|---|---|---|---|---|---|---|---|---|---|
| RF10 Recirculación | | | | | X | | | | | X |
| RF11 Condición del agua | | | | | | X | | | X | |
| RF12 Telemetría y consulta | | | | | | X | | X | | |
| RF13 Historial | | | X | | | | X | | | |
| RF14 Alertas | | | | | X | X | X | X | | |
| RF15 Estado seguro | | | X | X | X | | | X | | X |
| RF16 Reconexión | | | | | | | | X | | |
| RF17 No repetir comandos | | | X | | X | | | X | | X |
| RF18 Mantenimiento | | | | | | | | | X | |

CU01 sesión · CU02 configuración · CU03 alimento · CU04 RFID · CU05 agua · CU06 estado · CU07 historial · CU08 conexión · CU09 mantenimiento · CU10 parada.

### 12. Escenarios para validar los modelos

Los siguientes escenarios son verificaciones planeadas para la implementación; no constituyen evidencia de pruebas ya realizadas. Cada uno debe conservar sus logs, medidas o capturas para comparar el comportamiento observado con los modelos.

| Escenario | Comportamiento que deberá comprobarse |
|---|---|
| Porción normal para Chiquin o Tito | Cerrar dosificador por peso, guardar masa real y abrir acceso solo al UID seleccionado. |
| Otro tag frente al plato | Acceso cerrado y evento de rechazo; no cambiar el propietario de la porción. |
| Reenviar una orden y recargar la PWA | Una única dosificación para command_id; mostrar el mismo resultado. |
| Desconectar Wi-Fi durante el servido | Cierre local, resultado parcial o interrumpido y sincronización posterior sin otra dosis. |
| Apagar Laravel dejando Mosquitto activo | Vencimiento del lease y estado seguro aunque MQTT siga conectado. |
| Bajar el agua o invalidar su lectura | Parada local de bomba y bloqueo de encendido; recuperar nivel no reinicia la bomba. |
| Reiniciar durante una dosificación | Arranque seguro y comando interrumpido; inspección antes de nueva solicitud. |
| Guardar resultado con MySQL detenido | Evento pendiente en ESP32, sin confirmación falsa. Persistencia única al recuperar MySQL. |
| Porción fuera de tolerancia y alimento atascado | Conservar peso medido y causa. No corregir automáticamente con una dosis nueva. |
| Plato con alimento restante | Revalidar la misma sesión o requerir revisión; impedir otra preparación incompatible. |

## 13. Conclusiones

Los modelos establecen una cadena verificable desde la solicitud del usuario hasta el resultado medido y guardado. El backend administra permisos e historial, mientras el ESP32 conserva las decisiones inmediatas de peso, nivel y seguridad. La identificación RFID vincula una porción preparada con Chiquin o Tito sin confundir autorización de acceso con consumo real.

La máquina compuesta permite controlar alimento y agua de forma coordinada y responder a las fallas del enlace. Los resultados y comandos conservan identificadores para evitar repeticiones durante la recuperación. La validación del prototipo deberá concentrarse en calibración del pesaje, cierre de compuertas, ubicación del RFID, medición de niveles y respuesta segura ante desconexiones.

## 14. Referencias


[1] A. Navarrete Pérez, “Taller 4 Alternativas técnicas y viabilidad de ChiTito Smart Dispenser”, documento del proyecto, UABC, sep. 2026.

[2] OASIS, “MQTT Version 3.1.1”, OASIS Standard, 29 oct. 2014. Consulta: 7 oct. 2026. Especificación oficial de MQTT

[3] NXP Semiconductors, “MFRC522 Standard performance MIFARE and NTAG frontend”, Product data sheet, Rev. 3.9, 27 abr. 2016. Consulta: 7 oct. 2026. Datasheet del MFRC522

[4] DFRobot, “A02YYUW Waterproof Ultrasonic Distance Sensor”, SKU SEN0311, documentación técnica. Consulta: 7 oct. 2026. Documentación del A02YYUW

[5] DFRobot, “Gravity Analog TDS Sensor”, SKU SEN0244, documentación técnica. Consulta: 7 oct. 2026. Documentación del sensor TDS

[6] DFRobot, “Gravity Analog Turbidity Sensor”, SKU SEN0189, documentación técnica. Consulta: 7 oct. 2026. Documentación del sensor de turbidez

### 15. Ubicación de las imágenes entregadas

Los diagramas ya están integrados en este documento. También se entregan en archivos PNG de alta resolución dentro del archivo comprimido. La secuencia integral se presenta en dos partes dentro del reporte por legibilidad; en los archivos adjuntos se proporciona completa y también separada en sus dos partes. Los nombres de archivo y sus ubicaciones se indican a continuación.

| Archivo | Ubicación en el reporte |
|---|---|
| 01_contexto.png | Figura 1, después de actores y sistemas externos. |
| 02_actividad_alimento.png | Figura 2, sección 6, después del caso crítico CU03. |
| 03_actividad_agua.png | Figura 3, sección 6, después del caso crítico CU05. |
| 04_secuencia_integral.png | Figura 4, sección 7. Imagen completa de las dos continuaciones. |
| 05_estados_alimento.png | Figura 5, sección 8, región de porción y acceso RFID. |
| 06_estados_conectividad.png | Figura 6, sección 8, región de conexión y recuperación. |
| 07_estados_agua.png | Figura 7, sección 8, región de control de bomba. |